

CODE SPORT



Sandya Mannarswamy

In this month's column, I feature a number of interview questions related to the Linux operating system's internals and programming problems.

For the last couple of months, we have been discussing storage systems. A few of our student readers have written to me, asking for Linux-specific OS questions to be featured in the column. So let's take a break from our discussion on storage systems, which we will continue in next month's column.

Questions based on the internals of the Linux OS

1. How do you configure and custom-tailor your Linux kernel before building the sources?
2. In simple terms, the Linux kernel is nothing but a software program written in C (I wish it were that simple). Many of the common programming functionalities such as printing output, reading input and allocating memory are library routines in the standard C library, and any user application written in C makes calls to standard C library routines and utilises these routines. Why is it that the Linux kernel does not use the standard C library for such functions?
3. What is the significance of using the `'asm linkage'` qualifier on function definitions in the Linux kernel? Give a common example of when this qualifier is used.
4. What are the top and bottom halves of an interrupt handler? What work would you do in the top-half and what work would you do in the bottom half?
5. If, at any point in time, you want to find out how many interrupts of each type have been received in each CPU, how would you do so?
6. Recall that operating systems like Solaris/hp-ux/aix support threads at both the user and kernel level. How does Linux support threads at the kernel level?
7. What are Reserved Page Frames? Why are they required?
8. The Linux kernel has a zone allocator and a slab allocator. What is the zone allocator's unit of allocation and what is the slab allocator's unit of allocation? Why do we need two allocators?
9. What is the use of the `'sysenter'` instruction? How can it be used to speed up system call performance?
10. You are writing a new kernel module that requires the use of floating point instructions. You know that floating point operations in user applications are seamlessly supported. You are told that usage of floating point instructions inside the kernel code is not advisable. Is this correct? If so, why?
11. If you wanted to find out how long the system has been up since the last re-boot, how would you do so? What would your code look like if you are doing this from inside the kernel code itself?
12. Why is a virtual file system layer needed in the Linux kernel?
13. You have written a bad piece of code in which you are trying to de-reference a NULL pointer. If your code was part of a user application, what happens? If your piece of code was part of a kernel module, what happens?
14. What are the different IO schedulers available